Documentación de uso de IA

Colorfly Studio - Proyecto M1

Registro cronológico de prompts, decisiones y evidencia de desarrollo

Usé IA como guía de aprendizaje durante el desarrollo de Colorfly Studio. Este documento reúne consultas reales, respuestas breves y evidencias visuales para mostrar cómo fui planteando ideas, tomando decisiones, escribiendo el código y probando cada avance.

Frederick Bermúdez
Octubre de 2026

Propósito y forma de trabajo

Desde el inicio pedí que la IA funcionara como guía y no como sustituto del trabajo. Por eso el proceso se construyó en pasos: propuse objetivos, escribí y probé el código, compartí errores o resultados y pedí explicaciones antes de continuar.

Cada apartado conserva una cita breve del prompt, una cita de la orientación recibida y un comentario sobre la decisión que tomé o el cambio que implementé. Las capturas reproducen conversaciones reales y permiten relacionar cada idea con el avance que produjo en el proyecto.

Recorrido documentado

1. Acuerdo inicial de acompañamiento.
2. Estructura de las paletas temáticas.
3. Refactorización y variables CSS.
4. Copiado de colores y etiquetas accesibles.
5. Cambio de formato entre HEX y HSL.
6. Revisión de accesibilidad en el efecto de copiado.
7. Distribución responsive de las paletas.
8. Bloqueo de colores e historial local.
9. Validación de colores bloqueados.

1. Acuerdo inicial de acompañamiento

Prompt

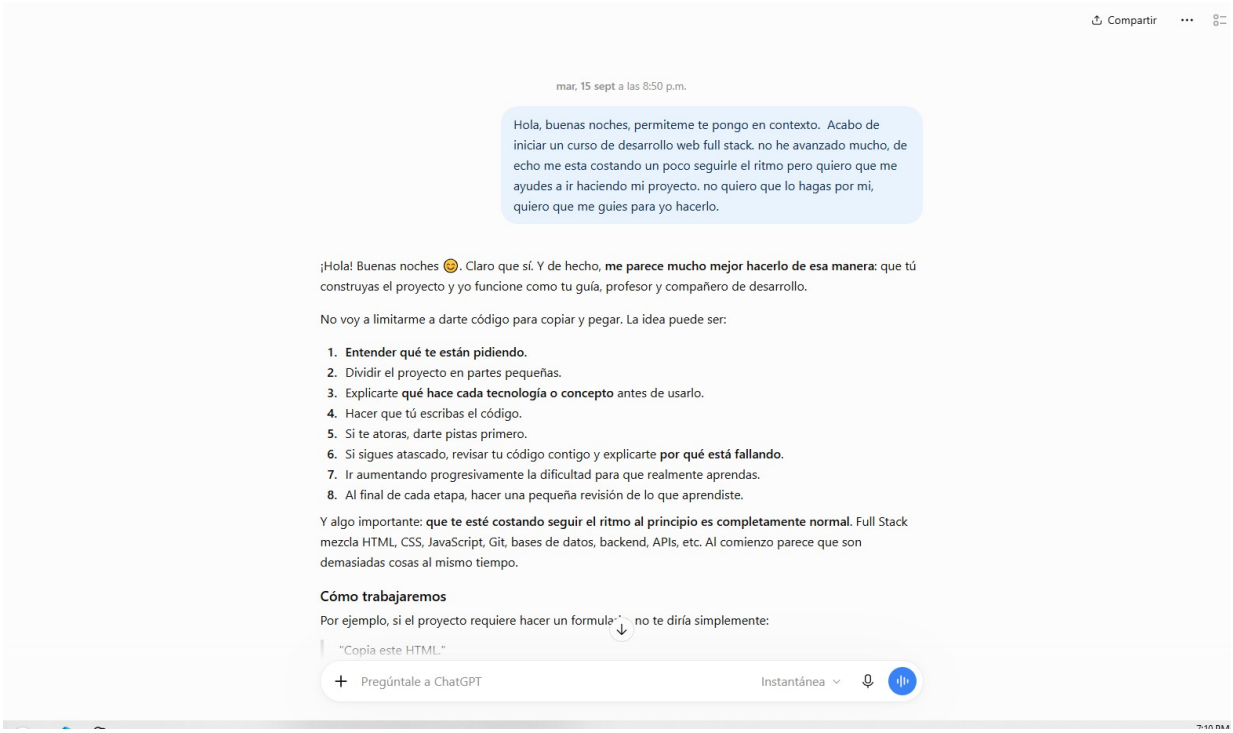
Acabo de iniciar un curso de desarrollo web full stack. No quiero que hagas mi proyecto por mí; quiero que me guíes para yo hacerlo.

Respuesta

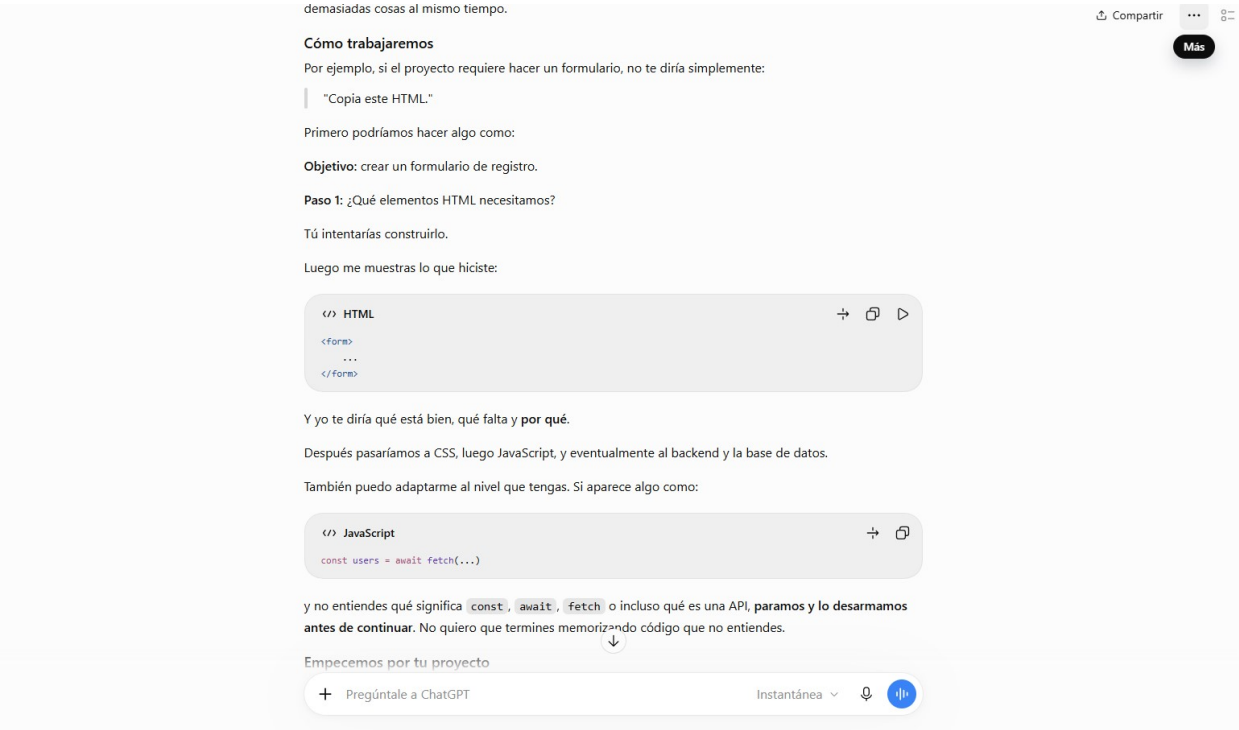
La idea es que tú construyas el proyecto y yo funcione como guía, profesor y compañero de desarrollo. Primero entendemos lo que piden, dividimos el proyecto en partes pequeñas y explicamos cada concepto antes de usarlo.

Comentario

Antes de iniciar Colorfly Studio dejé claro que quería aprender y construir el proyecto por mi cuenta. A partir de este acuerdo trabajé por etapas: planteé ideas, escribí el código, mostré resultados y pedí ayuda para entender o corregir lo necesario.



Captura 1. Acuerdo inicial sobre el rol de guía durante el proyecto.



Captura 2. Forma de trabajo acordada: intentar, revisar y entender antes de continuar.

2. Estructura de las paletas temáticas

Prompt

Quiero que cada paleta sea objeto con información, que por ejemplo diga un nombre, una breve descripción y en qué se suelen usar esos colores. Creo que debería usar un array para manejar los elementos asociados a esa lista.

Respuesta

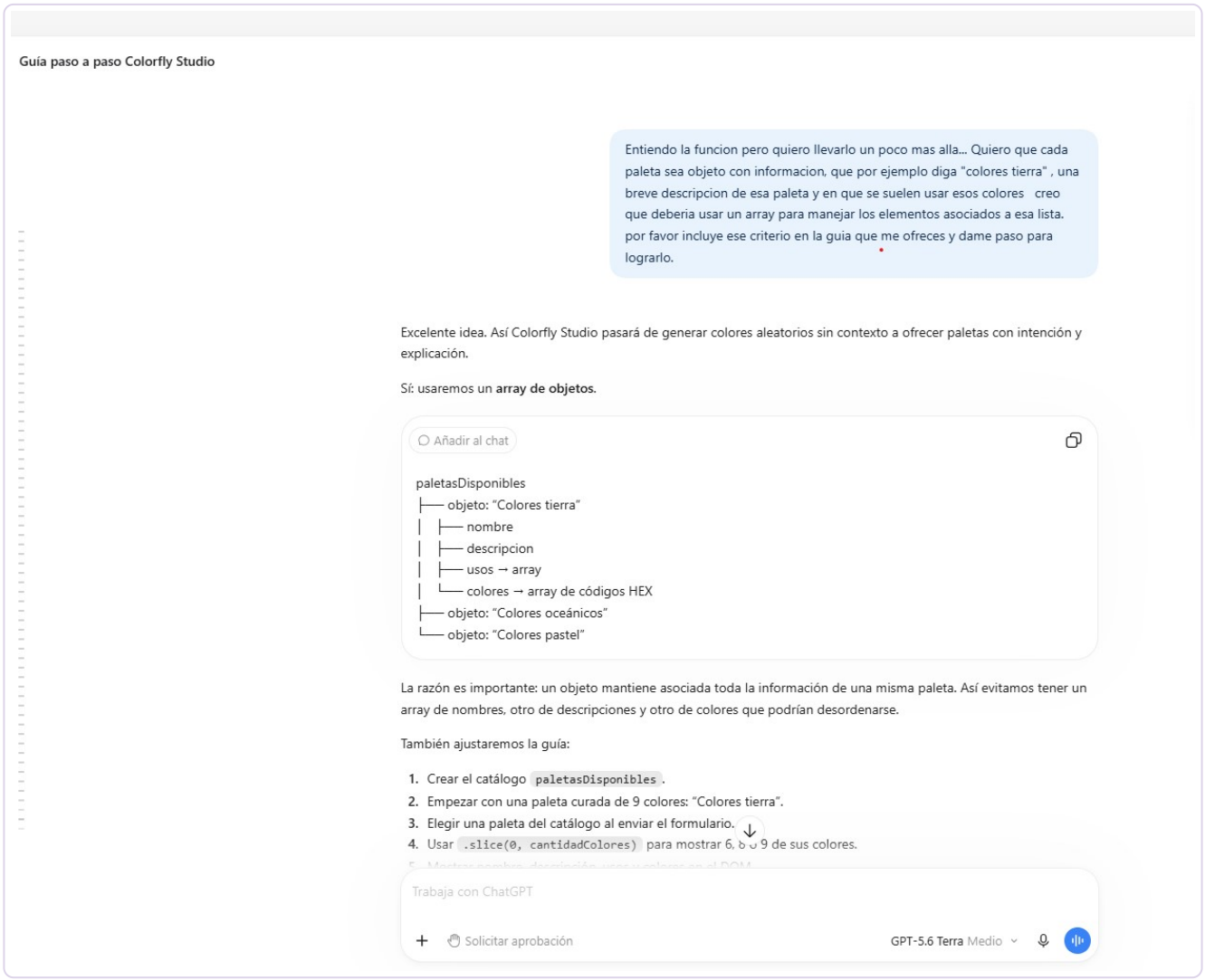
Conviene usar un array de objetos. Cada objeto puede contener nombre, descripción, usos y colores, para mantener relacionada toda la información de una misma paleta.

Comentario

Quise que las paletas temáticas no fueran solo listas de colores. A partir de esta conversación organicé paletasDisponibles como un array de objetos y añadí la información que luego aparece al generar una paleta temática. La explicación me ayudó a entender por qué un objeto era más adecuado que guardar datos en posiciones sueltas de un array.

Evidencia visual 2.1

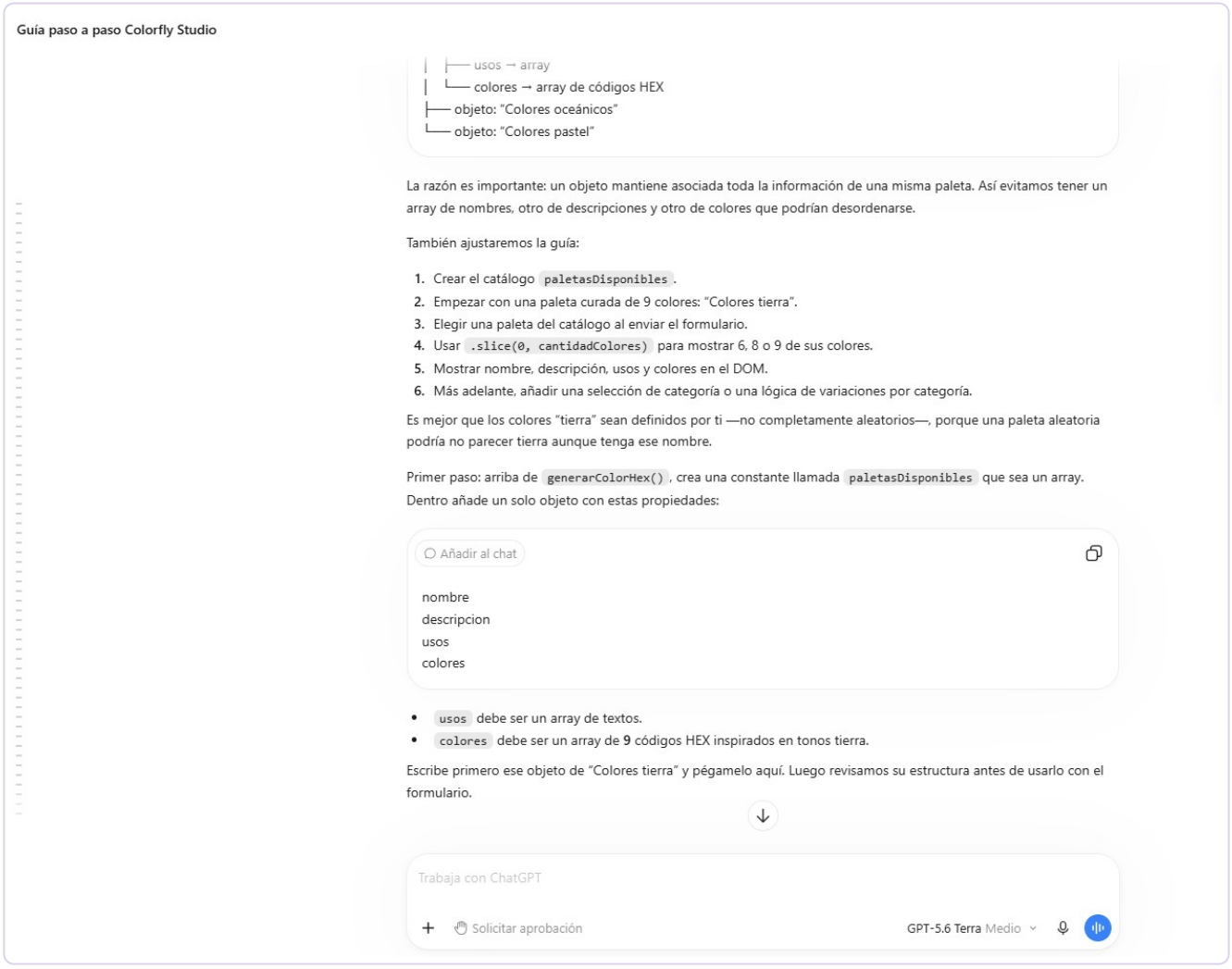
Conversación sobre la estructura de las paletas temáticas



Captura 12. Propuesta de organizar paletasDisponibles como un array de objetos.

Evidencia visual 2.2

Detalle de las propiedades de una paleta temática



Captura 13. Detalle de las propiedades y del proceso para construir la paleta temática.

3. Refactorización y variables CSS

Prompt

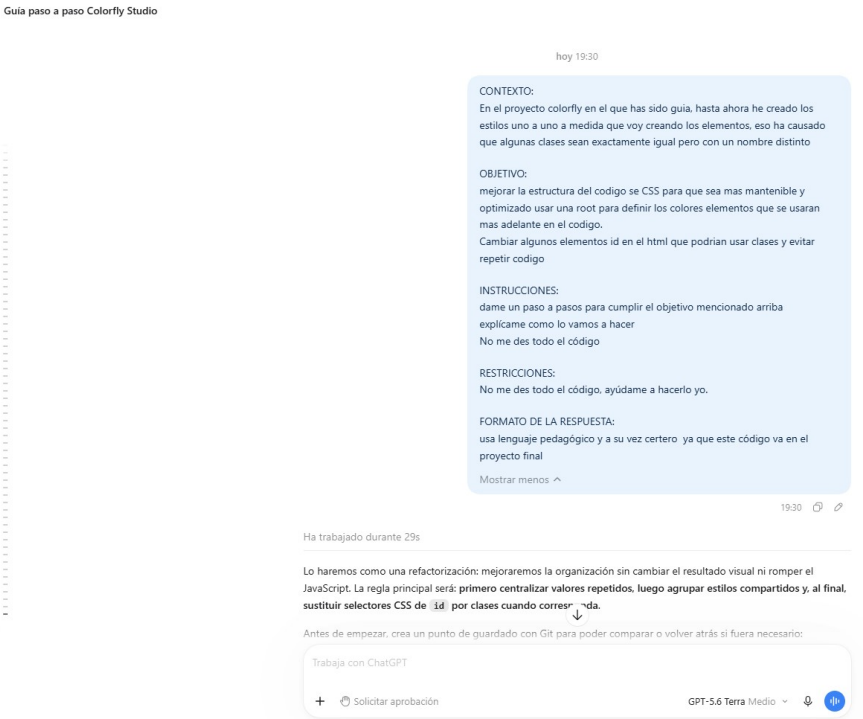
He creado los estilos uno a uno y algunas clases son exactamente igual pero con un nombre distinto. Quiero mejorar la estructura del CSS, usar root para definir colores y elementos reutilizables, y evitar repetir código. No me des todo el código; ayúdame a hacerlo yo.

Respuesta

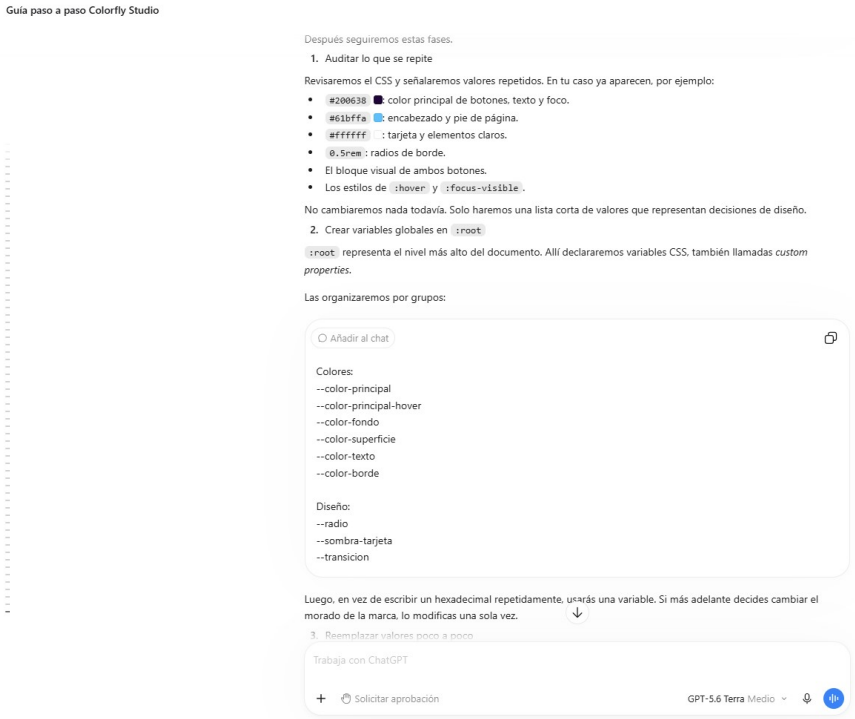
Lo haremos como una refactorización: primero centralizamos valores repetidos, luego agrupamos estilos compartidos y al final sustituimos selectores de id por clases cuando no afecten el JavaScript.

Comentario

Identifiqué que mis estilos ya tenían valores repetidos y pedí una guía sin recibir una solución completa para copiar. Organicé colores, radios, sombras y transiciones en :root; después fui reemplazando valores sin cambiar los id que JavaScript necesitaba.



Captura 3. Prompt estructurado con contexto, objetivo, restricciones y formato deseado.



Captura 4. Plan de auditoría y centralización de decisiones visuales en :root.

4. Copiado de colores y etiquetas accesibles

Prompt

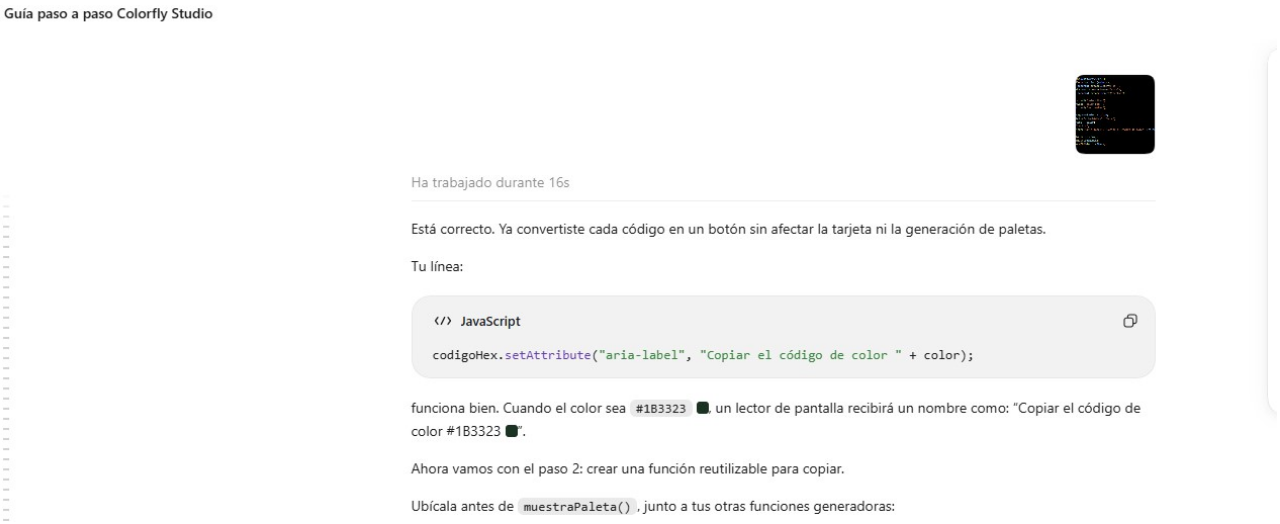
Quiero que cada código de color se pueda copiar al hacer clic. Elegí usar una función externa para hacerlo.

Respuesta

El código puede ser un botón con una etiqueta aria-label como ‘Copiar el código de color’ más el valor. Así la acción es clara para teclado y lector de pantalla, y la función de copiado puede reutilizarse en cada tarjeta.

Comentario

Convertí cada código de color en un botón y conecté su clic con una función reutilizable. También entendí que aria-label no cambia lo que se ve, sino la forma en que la acción se anuncia a personas que usan lector de pantalla.



Captura 5. Revisión del aria-label aplicado al botón que copia cada código de color.

5. Cambio de formato entre HEX y HSL

Prompt

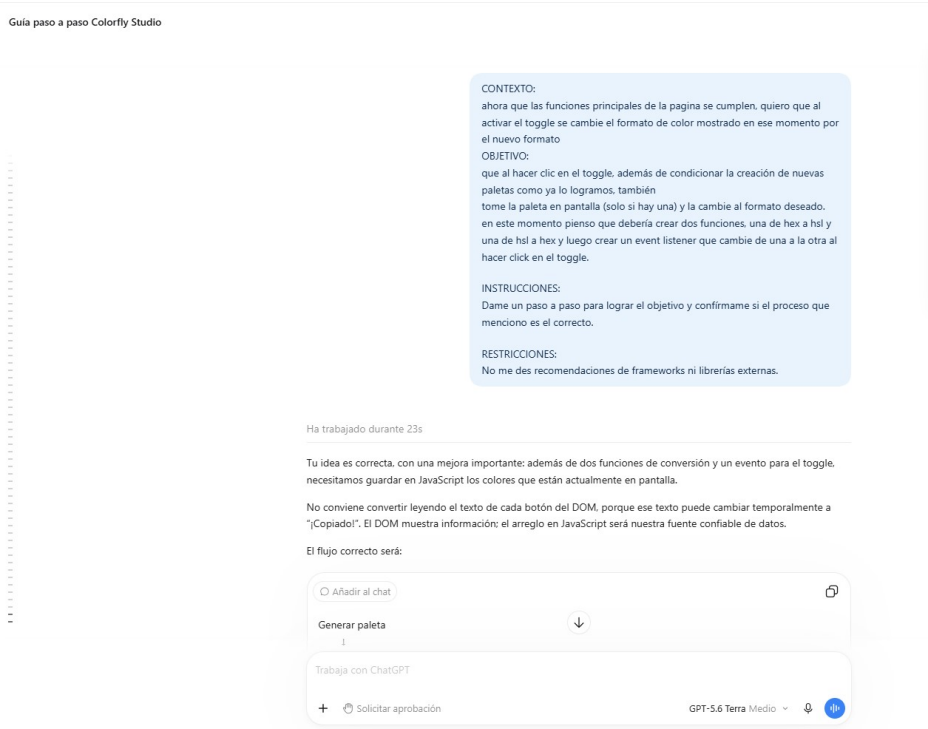
Quiero que al activar el toggle se cambie el formato de la paleta que está en pantalla. Pienso crear dos funciones, una de HEX a HSL y otra de HSL a HEX, y luego cambiar entre una y otra al hacer clic en el toggle.

Respuesta

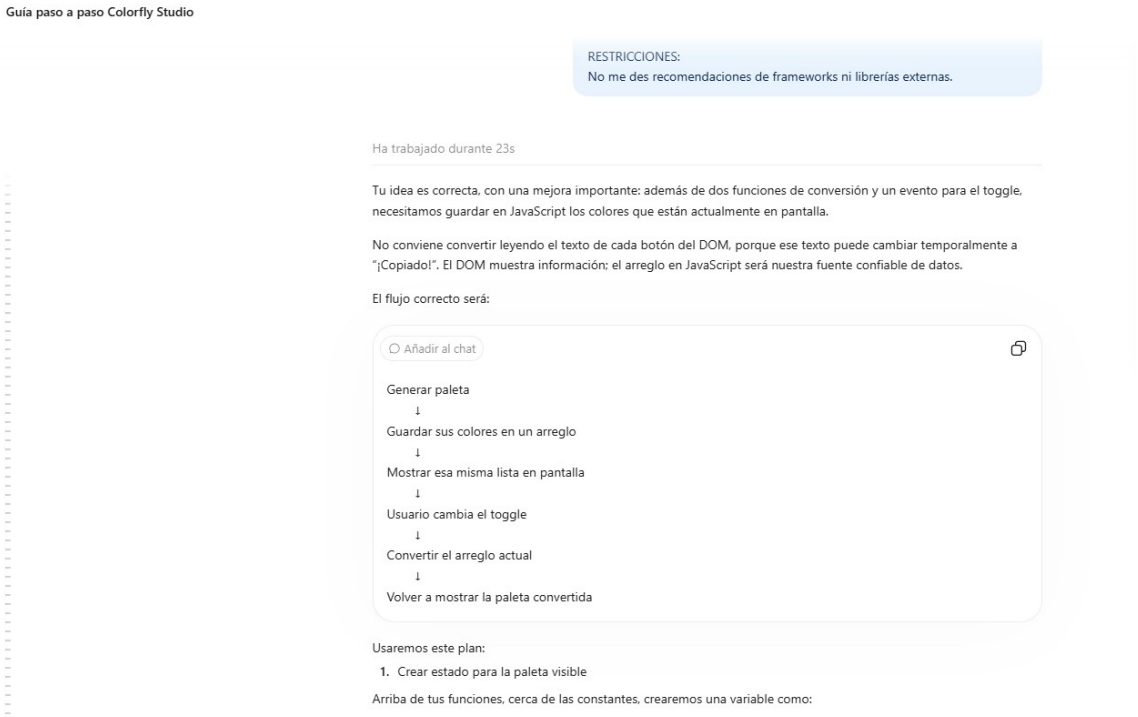
La idea es correcta, con una mejora importante: además de las funciones de conversión y el evento del toggle, necesitamos guardar en JavaScript los colores que están actualmente en pantalla. El arreglo será la fuente confiable de datos y luego se volverán a renderizar las tarjetas.

Comentario

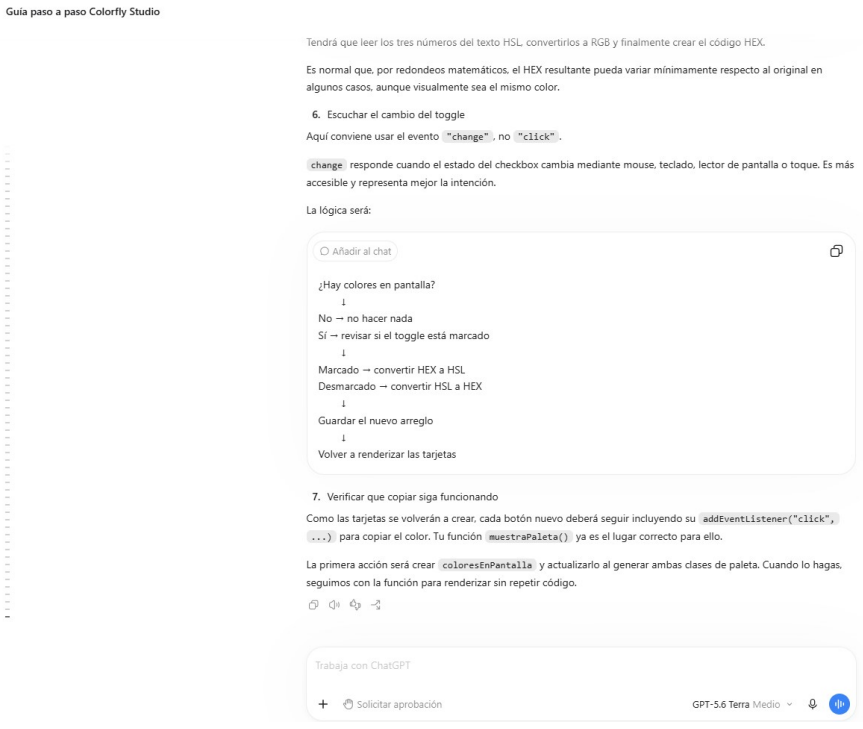
Propuse convertir la paleta visible, no solo usar el toggle para paletas nuevas. A partir de la orientación agregué coloresEnPantalla como estado de la aplicación, creé las conversiones y usé el evento change para que la interacción funcionara con mouse, teclado o toque.



Captura 6. Objetivo planteado para convertir la paleta visible entre formatos.



Captura 7. Flujo propuesto para conservar la paleta activa y volver a renderizarla.



Captura 8. Decisión de usar change y conservar la accesibilidad del toggle.

6. Revisión de accesibilidad en el efecto de copiado

Prompt

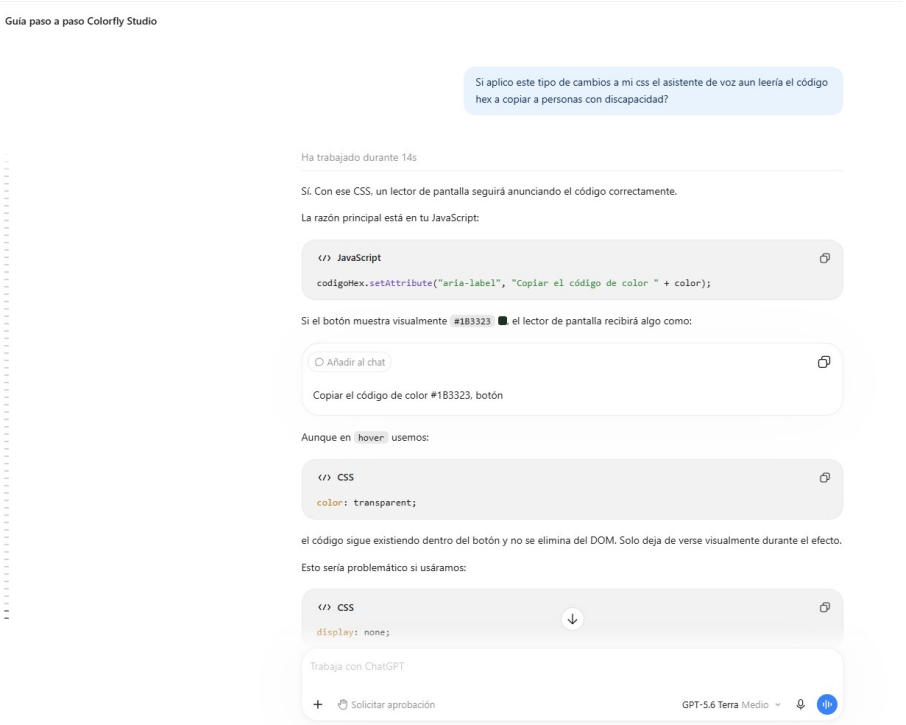
Si aplico este tipo de cambios a mi CSS, ¿el asistente de voz aún leería el código HEX a copiar a personas con discapacidad?

Respuesta

Sí. El lector de pantalla seguirá anunciando el código porque el contenido permanece dentro del botón y aria-label describe la acción. Ocultarlo visualmente con color transparente no equivale a eliminarlo del DOM; display: none sí impediría que se anuncie.

Comentario

Antes de dejar el efecto visual de ‘Copiar’, revisé si el código seguiría siendo accesible. Mantuve el valor dentro del botón, conservé la etiqueta ARIA y evité ocultar el contenido con display: none.



Captura 9. Consulta y explicación sobre el efecto visual de copiado y lectores de pantalla.

7. Distribución responsive según la cantidad de colores

Prompt

Quiero que los cuadrados se vean simétricos: tres filas de tres si se seleccionan 9 colores, dos filas de tres si se seleccionan 6 y dos filas de cuatro si se seleccionan 8. También quiero que la paleta de 8 no quede demasiado junta en teléfonos o pantallas pequeñas.

Respuesta

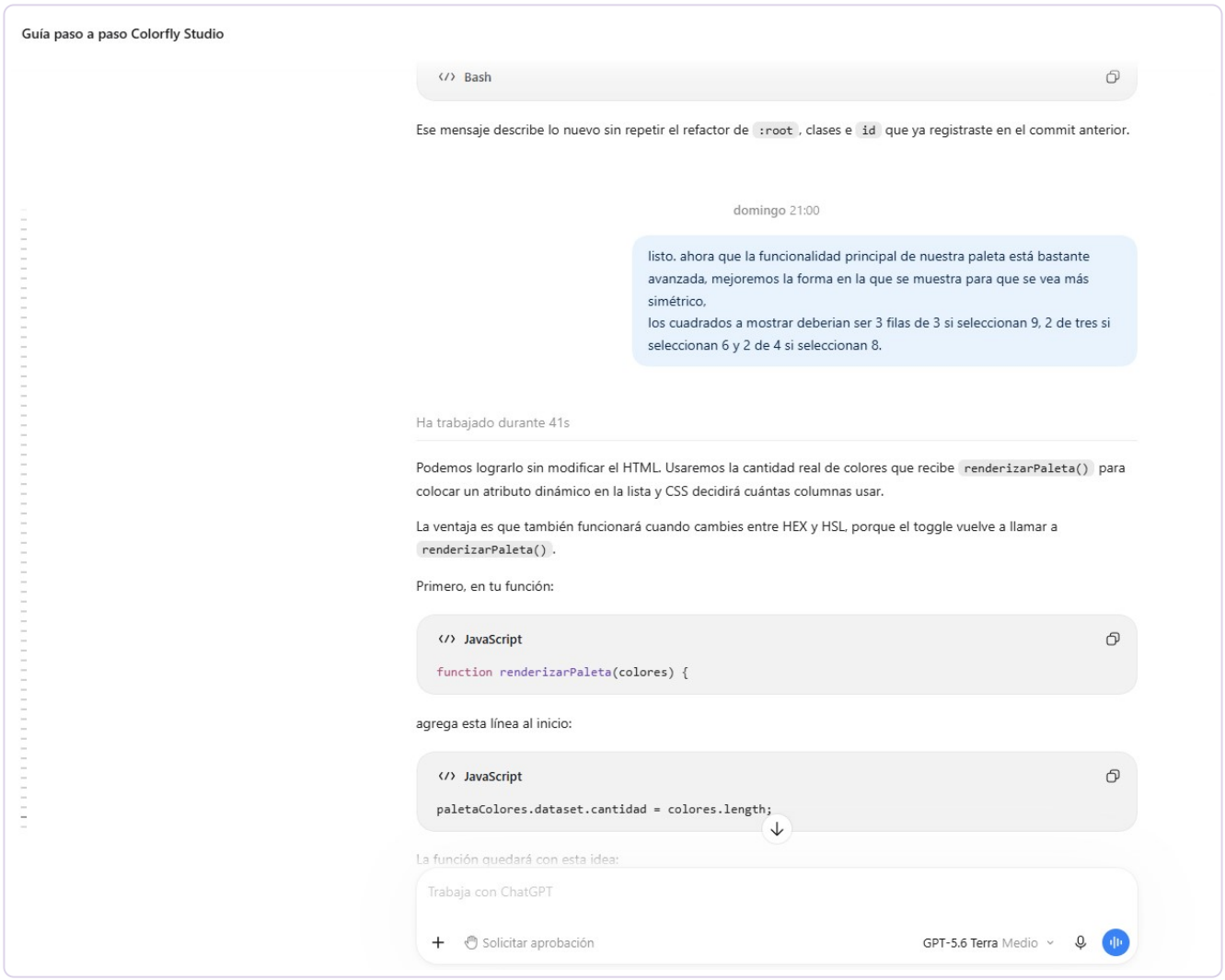
Podemos usar CSS Grid: tres columnas como distribución general, cuatro columnas cuando la lista tenga data-cantidad igual a 8 y una media query para que, en móvil, esa paleta pase a dos columnas.

Comentario

Después de implementar las tarjetas revisé que cada cantidad no se viera igual de bien con una cuadrícula genérica. Definí distribuciones distintas para 6, 8 y 9 colores y conecté la cantidad mostrada con data-cantidad, de modo que CSS ajustara la cuadrícula sin duplicar lógica de JavaScript.

Evidencia visual 7

Conversación sobre la distribución responsive



Captura 14. Uso de data-cantidad para adaptar la cuadrícula a 6, 8 o 9 colores.

8. Bloqueo de colores e historial local

Prompt

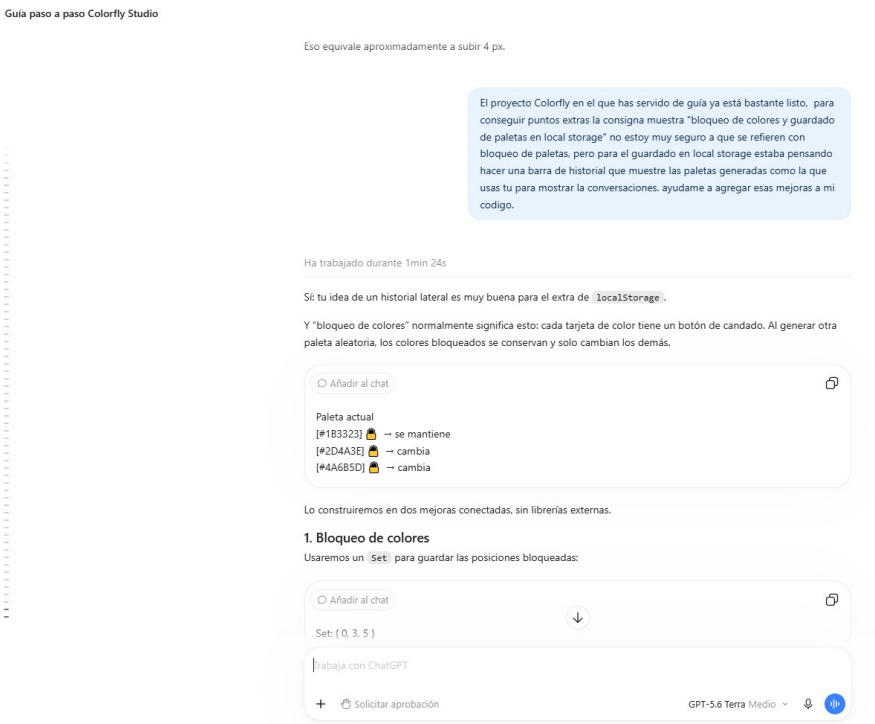
La consigna muestra ‘bloqueo de colores y guardado de paletas en local storage’. Para el guardado pensaba hacer una barra de historial que muestre las paletas generadas como la que usas para mostrar las conversaciones. Ayúdame a agregar esas mejoras a mi código.

Respuesta

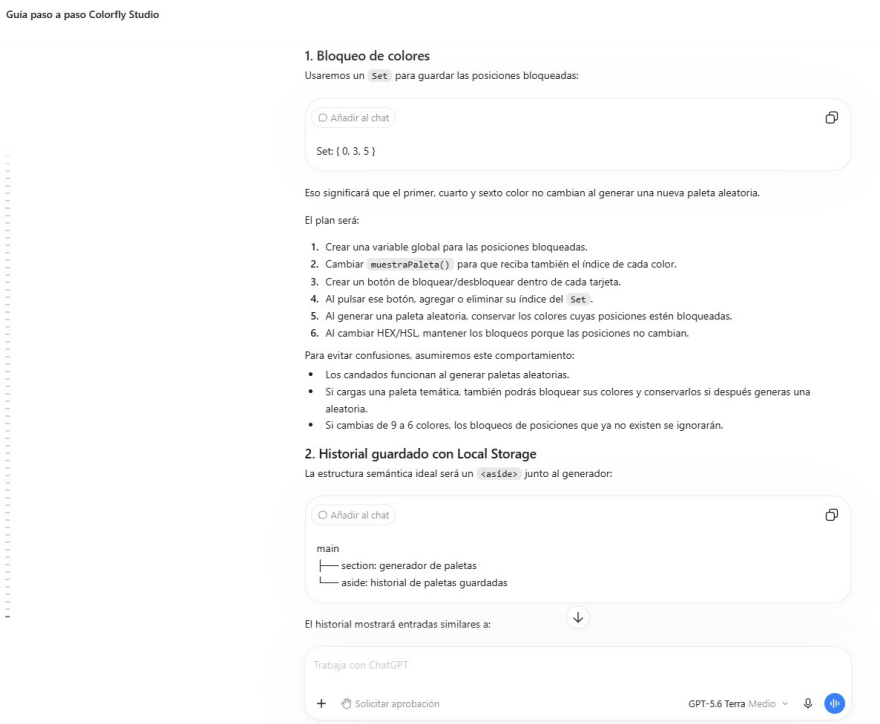
La idea de un historial lateral es adecuada para localStorage. Para el bloqueo usaremos un Set con las posiciones bloqueadas y, para el historial, un aside junto al generador. Los colores bloqueados se conservarán antes de completar una nueva paleta.

Comentario

Identifiqué dos mejoras que podían aportar puntos extra y propuse el historial lateral. A partir de esta conversación implementé el bloqueo por posiciones, el aside semántico, el guardado en localStorage y las acciones para cargar, eliminar y limpiar paletas guardadas.



Captura 10. Propuesta de añadir bloqueo de colores e historial persistente.



Captura 11. Plan técnico con Set para bloqueos y aside para el historial.

9. Validación de colores bloqueados

Prompt

Quiero que salga un mensaje de error si la cantidad de colores bloqueados es mayor que la cantidad de colores seleccionados. La paleta debe quedarse exactamente como está y el mensaje debe aparecer unos segundos y luego desaparecer.

Respuesta

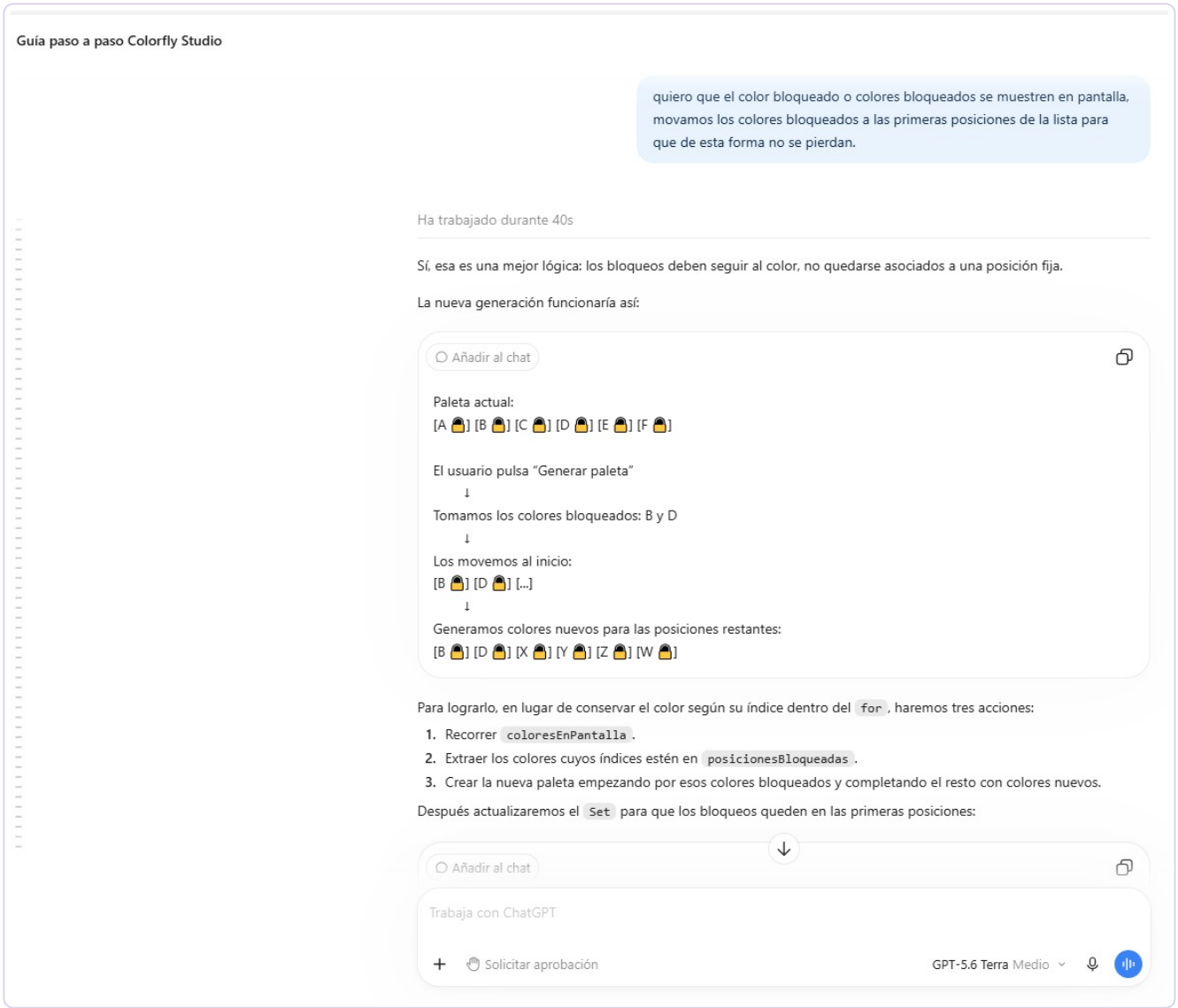
La validación debe ocurrir antes de generar o cambiar la paleta. Si falla, se muestra el error con mostrarError(...) y se termina la función con return. El temporizador se guarda en una variable y se usa clearTimeout antes de iniciar uno nuevo.

Comentario

Pensé en este caso límite mientras probaba el bloqueo de colores. No quería que una selección imposible eliminara una paleta válida. Agregué la validación antes de modificar el estado y mostré un mensaje temporal con role=alert para mantener una respuesta visible y accesible.

Evidencia visual 9.1

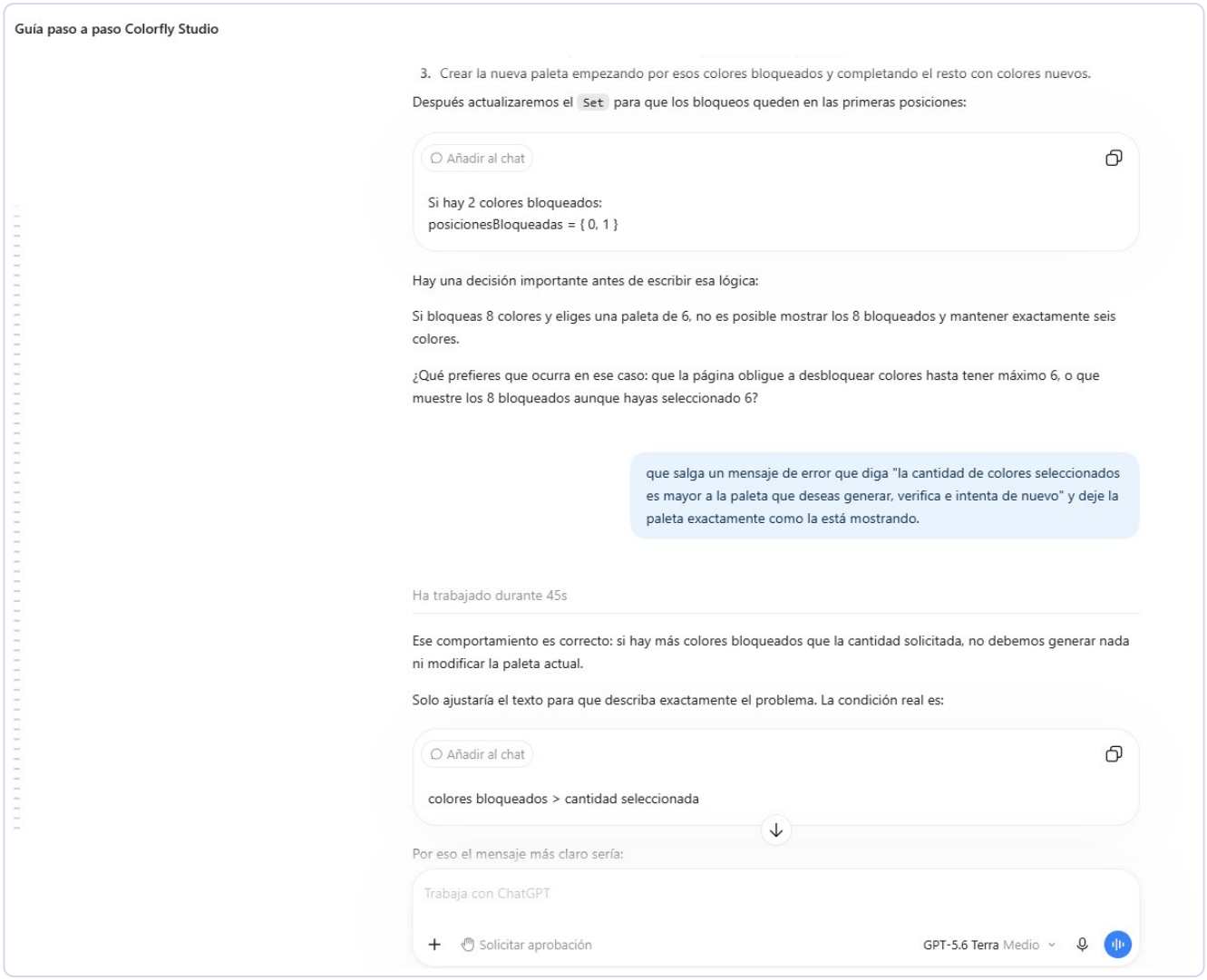
Conversación sobre el orden de los colores bloqueados



Captura 15. Decisión de mover los colores bloqueados a las primeras posiciones.

Evidencia visual 9.2

Conversación sobre la validación de colores bloqueados



Captura 16. Caso límite y decisión de conservar la paleta si hay demasiados bloqueos.

Cierre

El uso de IA en Colorfly Studio estuvo orientado a aprender, organizar decisiones y resolver dudas puntuales. Las evidencias muestran un proceso iterativo: propuse funcionalidades, pedí explicaciones, escribí y probé el código, revisé errores y ajusté la interfaz hasta llegar a la versión final.

Las decisiones técnicas descritas aquí se pueden contrastar con el README, el código fuente y el historial de commits del repositorio.